Finite-Blocklength Latency Minimization for Uplink and Downlink Communications: Per-User Scheduling Comparison for Unweighted and Inverse-CNR-Weighted Objectives

July 28, 2026

Abstract

This document compares per-user scheduling outcomes for the unweighted and inverse-CNR-weighted objectives in uplink and downlink payload completion. The search direction is fixed to descending so that the changes shown here reflect the objective choice rather than the search order. For each link, the *Training Only Method* is used as the benchmark and the *Training + Testing Method* is shown alongside it.

1 Scope and Naming

This note is schedule-focused. The terminology follows the companion reports:

- **Training Only Method**: direct online optimization during scheduling.
- **Training + Testing Method**: a separate training stage followed by testing-stage inference.
- **Unweighted objective**: each user's blockwise contribution is treated uniformly.
- **Inverse-CNR-weighted objective**: weaker users receive larger objective weight through inverse-CNR scaling.
- **Payload completion**: each user continues until its remaining payload is fully served.

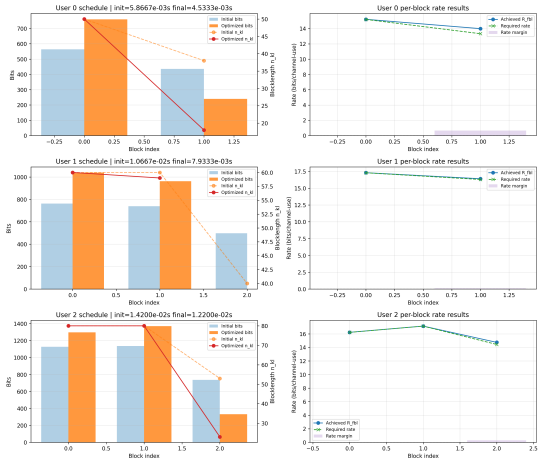
All figures and tables in this document use the descending-search objective comparison runs so that the main difference is the objective weighting itself.

2 Uplink

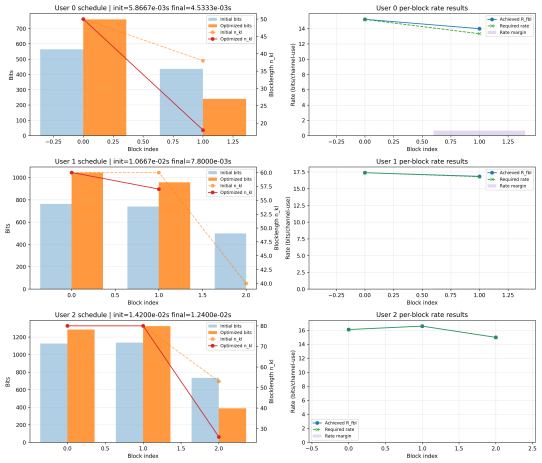
The uplink schedules remain structurally close across all four runs. The main effect of inverse-CNR weighting is a modest redistribution of sub-blocklengths inside the benchmark schedule, while the *Training + Testing Method* retains the same $(2, 3, 3)$ block pattern in both objective modes.

Metric	Training Only Unweighted	Training Only Inv.-CNR	Training + Testing Unweighted	Training + Testing Inv.-CNR
Blocks / user	2, 2, 3	2, 2, 3	2, 3, 3	2, 3, 3
Total n / user	68, 119, 183	68, 117, 186	67, 125, 182	67, 125, 182
Skipped blocks / user	0, 0, 0	0, 0, 0	0, 0, 0	0, 0, 0
Init. total	0.030733	0.030733	0.030733	0.030733
Final total	0.024667	0.024733	0.024933	0.024933
Red. (%)	19.7397	19.5228	18.8720	18.8720
Init. avg	0.010244	0.010244	0.010244	0.010244
Final avg	0.008222	0.008244	0.008311	0.008311
Init. async.	0.016667	0.016667	0.016667	0.016667
Final async.	0.015333	0.015733	0.015333	0.015333
Async. red. (%)	8.0000	5.6000	8.0000	8.0000
Init. SNR (dB)	8.1078	8.1078	8.1078	8.1078
Final SNR (dB)	8.4196	8.3587	8.3865	8.4089
Init. SINR (dB)	-3.3939	-3.3939	-3.3939	-3.3939
Final SINR (dB)	-3.3022	-3.3800	-3.3269	-3.3250

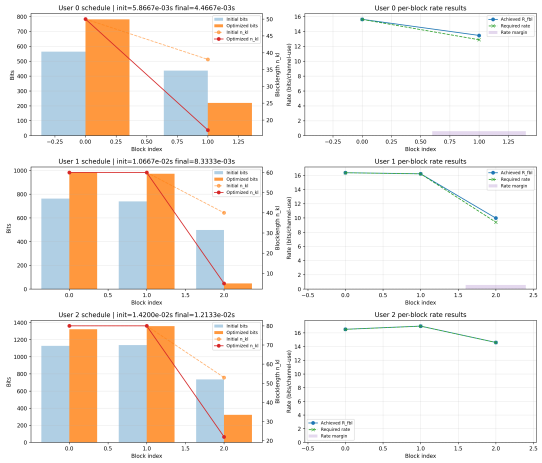
Table 1: Uplink payload-completion summary for the unweighted and inverse-CNR-weighted objectives.



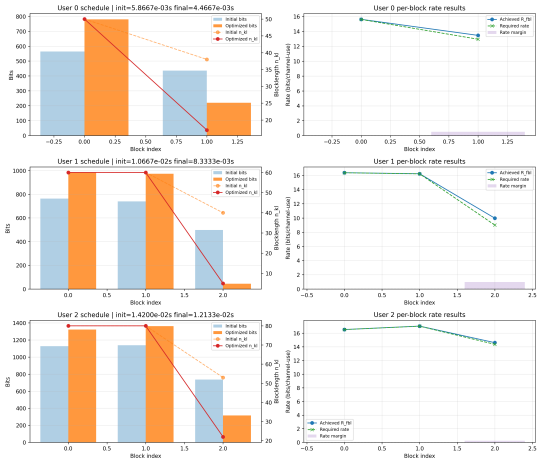
(a) Training Only Method, unweighted



(b) Training Only Method, inverse-CNR-weighted



(c) Training + Testing Method, unweighted



(d) Training + Testing Method, inverse-CNR-weighted

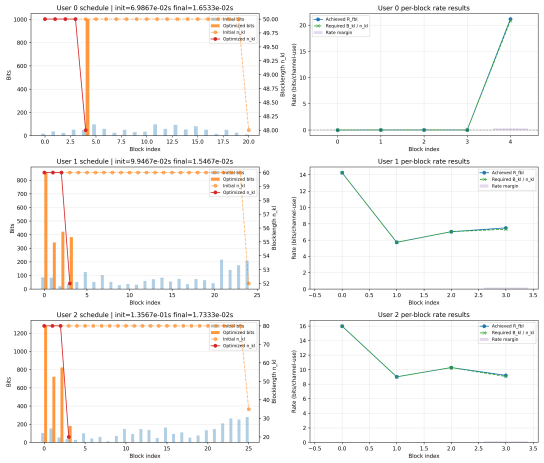
Figure 1: Uplink per-user schedule details for the unweighted and inverse-CNR-weighted objectives. The benchmark changes only slightly under weighting, and the *Training + Testing Method* keeps the same overall schedule shape in both modes.

3 Downlink

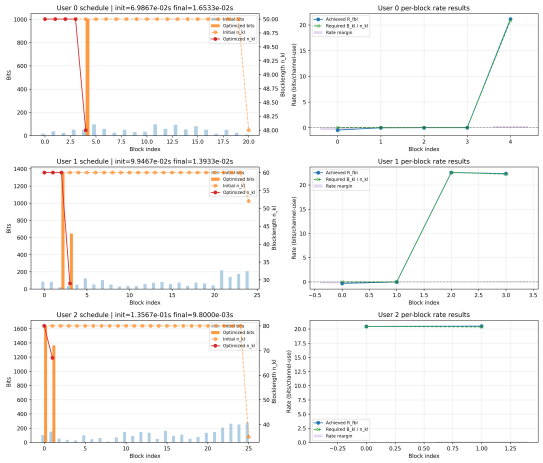
The downlink reacts more strongly to the objective choice. In the benchmark, inverse-CNR weighting shortens the schedule tail by collapsing user 2 from four blocks to two and reducing its total assigned sub-blocklength from 260 to 147. In the *Training + Testing Method*, the weighted run still has a longer schedule than the benchmark, but it improves both total latency and asynchronality relative to the unweighted case.

Metric	Training Only Unweighted	Training Only Inv.-CNR	Training + Testing Unweighted	Training + Testing Inv.-CNR
Blocks / user	5, 4, 4	5, 4, 2	9, 3, 7	9, 7, 4
Total n / user	248, 232, 260	248, 209, 147	437, 168, 530	436, 395, 273
Skipped blocks / user	4, 0, 0	4, 2, 0	7, 0, 3	7, 4, 0
Init. total	0.305000	0.305000	0.305000	0.305000
Final total	0.049333	0.040267	0.075667	0.073600
Red. (%)	83.8251	86.7978	75.1913	75.8689
Init. avg	0.101667	0.101667	0.101667	0.101667
Final avg	0.016444	0.013422	0.025222	0.024533
Init. async.	0.131600	0.131600	0.131600	0.131600
Final async.	0.003733	0.013467	0.048267	0.021733
Async. red. (%)	97.1631	89.7670	63.3232	83.4853
Init. SNR (dB)	8.0000	8.0000	8.0000	8.0000
Final SNR (dB)	14.1792	15.3203	9.3594	9.8556
Init. SINR (dB)	-2.8373	-2.8373	-2.8373	-2.8373
Final SINR (dB)	8.2492	18.6234	11.9746	13.0376

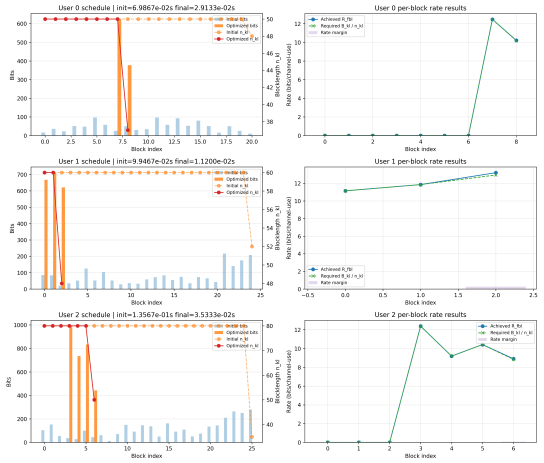
Table 2: Downlink payload-completion summary for the unweighted and inverse-CNR-weighted objectives.



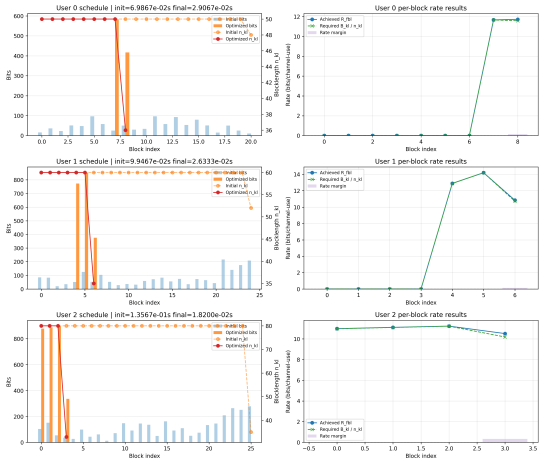
(a) Training Only Method, unweighted



(b) Training Only Method, inverse-CNR-weighted



(c) Training + Testing Method, unweighted



(d) Training + Testing Method, inverse-CNR-weighted

Figure 2: Downlink per-user schedule details for the unweighted and inverse-CNR-weighted objectives. The weighted benchmark becomes visibly shorter on user 2, while the weighted *Training + Testing Method* improves over the unweighted case but remains more fragmented than the benchmark.

4 Schedule Takeaways

In the uplink, inverse-CNR weighting changes the benchmark only marginally and does not change the *Training + Testing Method* schedule pattern at all. So the uplink objective comparison is mostly a fine redistribution effect rather than a structural scheduling shift.

In the downlink, the objective choice matters more. The benchmark benefits most clearly in final latency, while the *Training + Testing Method* gains both lower total latency and better synchronization under inverse-CNR weighting.